UNIVERSIDAD DEL CAUCA

LABORATORIO DE PROGRAMACIÓN ORIENTEDA A OBGETOS



DOCENTE: ING. RUBEM DARIO VARGAS YANDY

PRESENTADO POR:

CRISTIAN CAMILO MELO HORMAZA

LOREN NATALIA CAICEDO GRUESO

POPAYÁN, CAUCA
2026

TABLA DE CONTENIDO

1. Introducción..... 1

2. Objetivos..... 1

3. Descripción del sistema..... 1

4. Estructura y diseño de clases..... 5

5. Uso de Inteligencia Artificial.....7

6. Conclusiones..... 8

7. Anexos.....8

1. Introducción

Este informe presenta el trabajo realizado en el desarrollo de un sistema de gestión para bibliotecas. La aplicación fue programada en lenguaje C++ aplicando programación orientada a objetos, con el propósito de cubrir todos sus procesos básicos, desde el registro de autores, libros, estanterías y estudiantes, hasta la asignación de ejemplares, el control de préstamos y la generación de consultas e informes. Todo se diseñó con una navegación clara y segura, de modo que el usuario pueda rectificar datos o detener una operación en cualquier momento sin alterar la información guardada.

En este documento se describe paso a paso cómo está estructurado el sistema, qué clases y entidades lo componen, cómo funciona cada opción del menú, qué flujos sigue la información cuando se realizan las operaciones y qué pruebas se hicieron para asegurar que todo funcione como debe.

2. Objetivos

Objetivo general

Diseñar e implementar un sistema de gestión integral que permita registrar, consultar, actualizar y controlar los recursos bibliográficos, usuarios y transacciones de préstamo de la biblioteca de la Universidad del Cauca.

Objetivos específicos

•Modelar las entidades del dominio mediante clases con atributos, métodos y relaciones coherentes.

•Implementar flujos guiados que garanticen la consistencia en el registro de préstamos.

• Facilitar el uso del sistema con menús claros, validación de datos y opción de cancelar en cualquier momento.

3. Descripción del sistema

El sistema desarrollado tiene como finalidad administrar de forma completa y ordenada toda la información que maneja una biblioteca universitaria. Para lograrlo, se organizó el trabajo en distintas entidades, cada una con su propia función y características bien definidas.

En primer lugar está el Autor, que almacena el nombre, la fecha de nacimiento y la nacionalidad de cada persona creadora de las obras; esta información queda vinculada directamente a cada libro que se ingresa al sistema, permitiendo identificar claramente su procedencia.

Luego se encuentra el Libro, que registra el título, el número de ISBN, el año de publicación y la referencia al autor correspondiente, con la posibilidad de marcar la obra como de autor anónimo si no se cuenta con esa información.

Cada ejemplar se organiza físicamente a través del Estante, que lleva su nombre y mantiene la relación de los libros que contiene junto con la cantidad de ejemplares disponibles de cada uno. Los estantes a su vez se agrupan dentro de la Biblioteca, que registra su nombre, la facultad o unidad académica a la que pertenece y el conjunto de estanterías que tiene asignadas. De esta forma se sabe en qué lugar se encuentra cada recurso con precisión.

Por el lado de los usuarios, se cuenta con el Estudiante, al que se le asigna un código identificativo y su nombre completo, datos que permiten reconocerlo de manera única en el sistema. Cuando un estudiante toma un libro, se genera un registro de Préstamo, que vincula de forma exacta a la persona con la obra, el estante y la biblioteca correspondientes, dejando así constancia completa de la operación. Todo este conjunto de información se reúne y se coordina desde la clase Sistema, que actúa como núcleo central, asegurando que todos los datos estén conectados y funcionen de manera integrada.

En cuanto a su funcionamiento, el programa presenta un menú principal con catorce opciones que cubren cada uno de los procesos necesarios: desde el registro de autores, libros, estanterías, bibliotecas y estudiantes, hasta la asignación de ejemplares, la consulta de disponibilidad, la actualización de cantidades, el control de préstamos y la generación de reportes. El sistema valida antes de cada operación que existan los registros requeridos, ofrece la posibilidad de cancelar en cualquier paso y finaliza su ejecución cuando el usuario selecciona la opción 0.

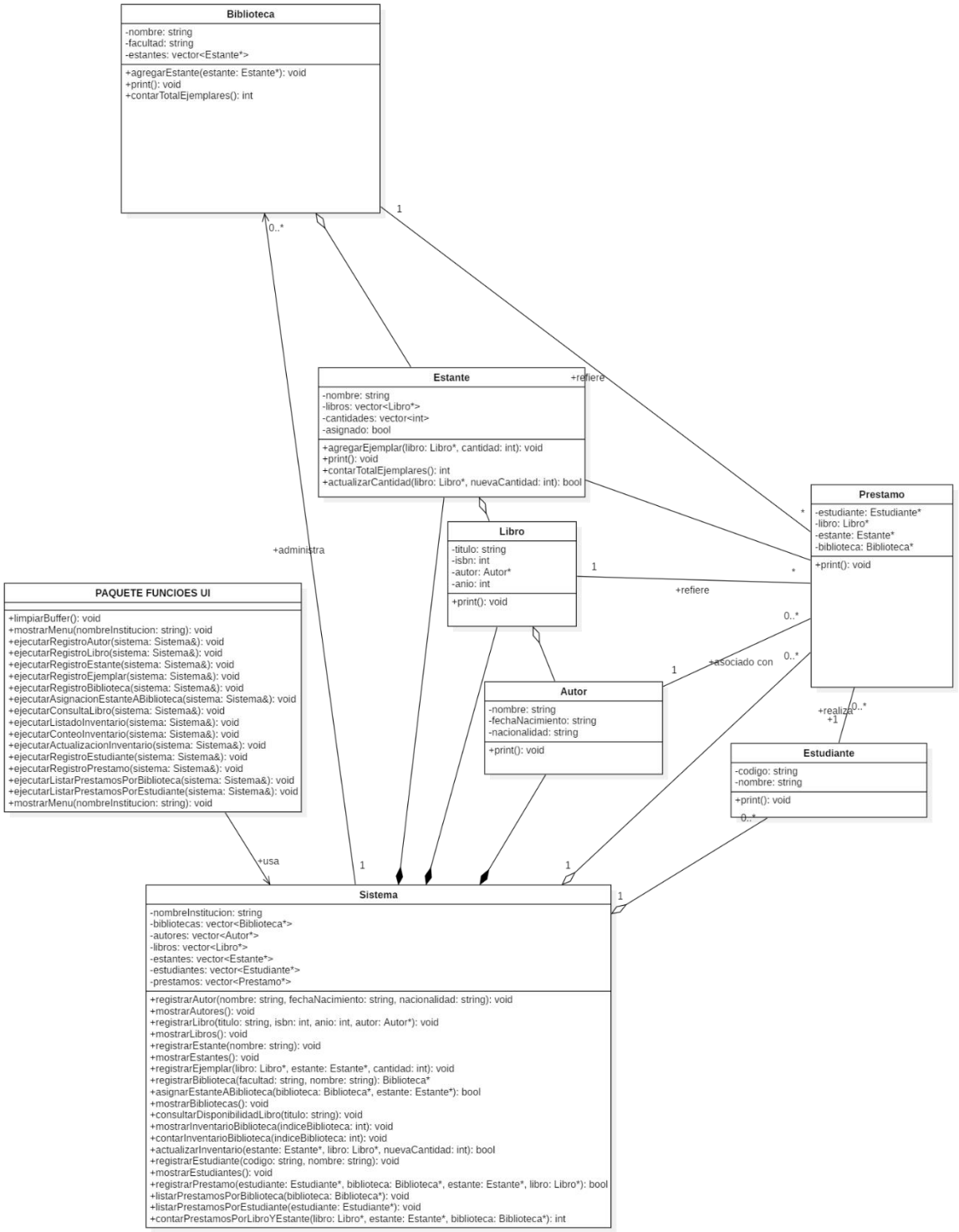
4. Estructura y diseño de clases

El diagrama de clases adjunto refleja la estructura completa del sistema, organizada en entidades, la clase controladora y las funciones de interfaz de usuario.

En la base del diagrama se encuentra la clase Sistema, que funciona como núcleo central. En ella se declaran lo siguiente bibliotecas, autores, libros, estantes, estudiantes y prestamos, cada uno almacenado en su respectiva lista. También contiene todos los métodos que operan sobre la información de registrar autores, libros, estantes, bibliotecas y estudiantes; asignar estantes a bibliotecas; consultar disponibilidad de libros; actualizar cantidades de inventario; registrar préstamos; y listar los préstamos filtrados por biblioteca o por estudiante. Es decir que aquí donde se coordinan todas las operaciones y se mantiene la consistencia de los datos conectadas directamente a la clase Sistema aparecen las entidades principales: Autor, con atributos para el nombre, la fecha de nacimiento y la nacionalidad, además de su método de impresión; Libro, que lleva título, ISBN, año y una referencia al autor; Estante, con su nombre, la lista de libros que contiene y las cantidades disponibles, además de métodos para agregar ejemplares, mostrar su contenido y actualizar existencias; Biblioteca, con nombre, facultad y la colección de estantes asignados, incluyendo la función para contar el total de ejemplares; Estudiante, identificado por código y nombre; y Prestamo, que reúne referencias al estudiante, al libro, al estante y a la biblioteca involucrados en la transacción.

Las relaciones entre entidades se aprecian claramente en el diagrama: un libro se asocia con un autor; un estante puede contener varios libros; una biblioteca agrupa varios estantes; y un préstamo vincula a las cuatro entidades —estudiante, libro, estante y biblioteca— para que quede registrado con toda su información.

A la izquierda del diagrama se observa el Paquete Funciones UI, que agrupa todas las rutinas encargadas de la interacción con el usuario. Estas funciones no manejan directamente los datos, sino que reciben las entradas, las validan y llaman a los métodos correspondientes de la clase Sistema. Se incluye la función para limpiar el búfer de entrada y cada una de las funciones que ejecutan las opciones del menú, manteniendo así separada la lógica del programa de la presentación por pantalla.


Biblioteca
-nombre: string
-facultad: string
-estantes: vector<Estante*>
+agregarEstante(estante: Estante*): void
+print(): void
+contarTotalEjemplares(): int
Estante
+refiere
-nombre: string
-libros: vector<Libro*>
-cantidades: vector<int>
-asignado: bool
+agregarEjemplar(libro: Libro*, cantidad: int): void
+print(): void
+contarTotalEjemplares(): int
+actualizarCantidad(libro: Libro*, nuevaCantidad: int): bool
Libro
-titulo: string
-isbn: int
-autor: Autor*
-anio: int
+print(): void
Prestamo
-estudiante: Estudiante*
-libro: Libro*
-estante: Estante*
-biblioteca: Biblioteca*
+print(): void
PAQUETE FUNCIOES UI
+limpiarBuffer(): void
+mostrarMenu(nombreInstitucion: string): void
+ejecutarRegistroAutor(sistema: Sistema&): void
+ejecutarRegistroLibro(sistema: Sistema&): void
+ejecutarRegistroEstante(sistema: Sistema&): void
+ejecutarRegistroEjemplar(sistema: Sistema&): void
+ejecutarRegistroBiblioteca(sistema: Sistema&): void
+ejecutarAsignacionEstanteABiblioteca(sistema: Sistema&): void
+ejecutarConsultaLibro(sistema: Sistema&): void
+ejecutarListadoInventario(sistema: Sistema&): void
+ejecutarConteoInventario(sistema: Sistema&): void
+ejecutarActualizacionInventario(sistema: Sistema&): void
+ejecutarRegistroEstudiante(sistema: Sistema&): void
+ejecutarRegistroPrestamo(sistema: Sistema&): void
+ejecutarListarPrestamosPorBiblioteca(sistema: Sistema&): void
+ejecutarListarPrestamosPorEstudiante(sistema: Sistema&): void
+mostrarMenu(nombreInstitucion: string): void
Autor
-nombre: string
-fechaNacimiento: string
-nacionalidad: string
+print(): void
Estudiante
-codigo: string
-nombre: string
+print(): void
Sistema
-nombreInstitucion: string
-bibliotecas: vector<Biblioteca*>
-autores: vector<Autor*>
-libros: vector<Libro*>
-estantes: vector<Estante*>
-estudiantes: vector<Estudiante*>
-prestamos: vector<Prestamo*>
+registrarAutor(nombre: string, fechaNacimiento: string, nacionalidad: string): void
+mostrarAutores(): void
+registrarLibro(titulo: string, isbn: int, anio: int, autor: Autor*): void
+mostrarLibros(): void
+registrarEstante(nombre: string): void
+mostrarEstantes(): void
+registrarEjemplar(libro: Libro*, estante: Estante*, cantidad: int): void
+registrarBiblioteca(facultad: string, nombre: string): Biblioteca*
+asignarEstanteABiblioteca(biblioteca: Biblioteca*, estante: Estante*): bool
+mostrarBibliotecas(): void
+consultarDisponibilidadLibro(titulo: string): void
+mostrarInventarioBiblioteca(indiceBiblioteca: int): void
+contarInventarioBiblioteca(indiceBiblioteca: int): void
+actualizarInventario(estante: Estante*, libro: Libro*, nuevaCantidad: int): bool
+registrarEstudiante(codigo: string, nombre: string): void
+mostrarEstudiantes(): void
+registrarPrestamo(estudiante: Estudiante*, biblioteca: Biblioteca*, estante: Estante*, libro: Libro*): bool
+listarPrestamosPorBiblioteca(biblioteca: Biblioteca*): void
+listarPrestamosPorEstudiante(estudiante: Estudiante*): void
+contarPrestamosPorLibroYEstante(libro: Libro*, estante: Estante*, biblioteca: Biblioteca*): int
1
0..*
+administra
+refiere
+asociado con
+realiza
+1
+usa

5. Uso de Inteligencia Artificial

El profesor autorizó el uso de inteligencia artificial como herramienta de apoyo durante la elaboración de este trabajo, indicando que debemos explicar claramente en qué partes se recurrió a ella. A continuación se detalla todo lo que se realizó con su ayuda:

1. Estructuración del menú principal

Se solicitó ayuda para crear la función que muestra el menú con las catorce opciones del sistema y la opción 0 para salir. También se generó la función limpiarBuffer(), que soluciona los problemas que surgen al leer datos del teclado. Con esto se logró una navegación clara y ordenada desde el inicio del programa.

2. Flujo para registrar préstamos

Se recibió apoyo para diseñar el proceso completo de registro de un préstamo, paso a paso: primero se selecciona el estudiante, luego la biblioteca, después el estante y por último el libro. En cada etapa el sistema verifica que existan los registros necesarios y permite cancelar la operación escribiendo 0, sin perder la información guardada.

3. Consultas de préstamos

Se usó la herramienta para construir las dos funciones de reportes: una que muestra los préstamos realizados en una biblioteca elegida y otra que muestra los libros que tiene prestados un estudiante específico. Ambas verifican que haya registros antes de continuar y avisan si no hay datos para mostrar.

4. Registro de autores y libros

Se contó con apoyo para diseñar el registro de autores, permitiendo dejar vacíos la fecha de nacimiento o la nacionalidad; en ese caso el sistema asigna automáticamente "No registrado". Para los libros, se implementó la opción de elegir un autor ya registrado o dejar la obra como "Anónimo" si no se cuenta con esa información.

5. Actualización de inventario

Se recibió ayuda para crear la función que permite modificar la cantidad de ejemplares en un estante. El sistema muestra primero los libros con sus cantidades actuales, deja seleccionar el que se va a cambiar, valida que la nueva cantidad no sea negativa y aplica el cambio. En cualquier momento se puede cancelar la operación con 0.

Es importante aclarar que la inteligencia artificial sirvió como apoyo para escribir y organizar el código, pero todo fue revisado, comprendido y verificado por los integrantes del equipo. La estructura de clases, el diseño del sistema y la lógica general fueron definidos por nosotros, y cada parte del trabajo fue revisada para asegurar que funcionara correctamente.

6. Conclusiones

El sistema desarrollado cubre por completo los procesos básicos de gestión que requiere una biblioteca: el registro de autores, libros, estanterías, bibliotecas y estudiantes, así como el control de inventario y la administración de préstamos. Todo está organizado mediante programación orientada a objetos, lo que mantiene el código claro, ordenado y fácil de entender o ampliar en el futuro.

La estructura diseñada garantiza que la información se mantenga consistente: cada vez que se presta un libro, se actualizan automáticamente las existencias, y el sistema verifica en cada paso que existan los registros necesarios antes de continuar. Además, la interfaz de menús y la posibilidad de cancelar cualquier operación hacen que el programa sea seguro y sencillo de usar, sin riesgo de perder la información ya guardada.

Como trabajo académico, este proyecto permitió aplicar los conocimientos de programación orientada a objetos en una situación real. Se logró construir una herramienta funcional que responde a lo que se espera de un sistema de gestión, con validaciones, reportes y un flujo de trabajo ordenado. Entre las mejoras que se podrían incorporar más adelante se encuentran guardar la información en archivos para que no se pierda al cerrar el programa, controlar las fechas de devolución o diseñar una interfaz gráfica más amigable.

7.Anexos

- **Bitácora de Pronts e Interacción con Inteligencia Arficial.pdf**
- **Diagrama de clases.png**
- **Código del programa: Main.cpp**